

07 System of Linear Equations



7.1 Linear Equations in Two Variables

From the Chicken-Rabbit Puzzle to Matrix Form: Algebra, Geometry, and Matrices, Three Views on Linear Systems

Linear Algebra Made Easy and Visual | Open-source code & drafts: <https://github.com/Visualize-ML>

Core Skills You Will Master

1

Turn a real-world puzzle like chicken-rabbit-in-a-cage into a system of linear equations

2

Algebraic methods: solve unknowns by substitution/elimination and by the inverse matrix

3

Geometric view: each linear equation in two variables is a line; the solution is where lines meet

4

Read off from a graph whether a system has a unique solution, no solution, or infinitely many

5

Matrix form $\mathbf{Ax}=\mathbf{b}$: a compact, efficient way to express a linear system

6

Use the determinant $\det(\mathbf{A})$ to test whether a system has a unique solution

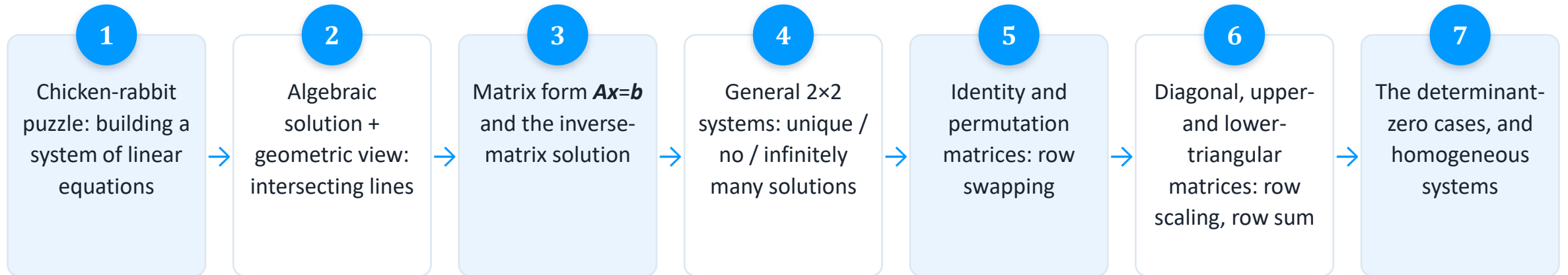
7

Understand three elementary row operations: row swapping, row scaling, row sum

8

Solve systems in code with `numpy.linalg.inv()` and `numpy.linalg.solve()`

From the Chicken-Rabbit Puzzle to a General Matrix Solution



Sun Zi's Classic: The Chicken-Rabbit Puzzle

"There are chickens and rabbits in the same cage, 35 heads above and 94 feet below — how many chickens and rabbits are there?"

Let x_1 be the number of chickens, x_2 the number of rabbits: the head count and the foot count each give one linear equation, and together they form a system

Linear Equation in Two Variables

$$a_1x_1 + a_2x_2 = b$$

- An algebraic equation with two unknowns x_1, x_2 , each raised to at most the first power; a_1, a_2 are any real coefficients, b any real constant
- Head-count equation: $x_1+x_2=35$; foot-count equation: $2x_1+4x_2=94$ — together these two equations form a system of linear equations

Solving the Chicken-Rabbit Puzzle by Substitution

$$x_1 + x_2 = 35, 2x_1 + 4x_2 = 94 \Rightarrow x_1 = 23, x_2 = 12$$

- Divide the foot-count equation by 2 to get $x_1 + 2x_2 = 47$; subtract the head-count equation $x_1 + x_2 = 35$ to eliminate x_1 , giving $x_2 = 12$, then $x_1 = 23$
- The cage holds 23 chickens and 12 rabbits — an implicit condition: both x_1 and x_2 must be non-negative integers

Figure 1 · Different Coefficients, Different Line Shapes

- $a_2=0$ gives a vertical line; $a_1=0$ gives a horizontal line; when both a_1 and a_2 are nonzero, the line has a nonzero slope

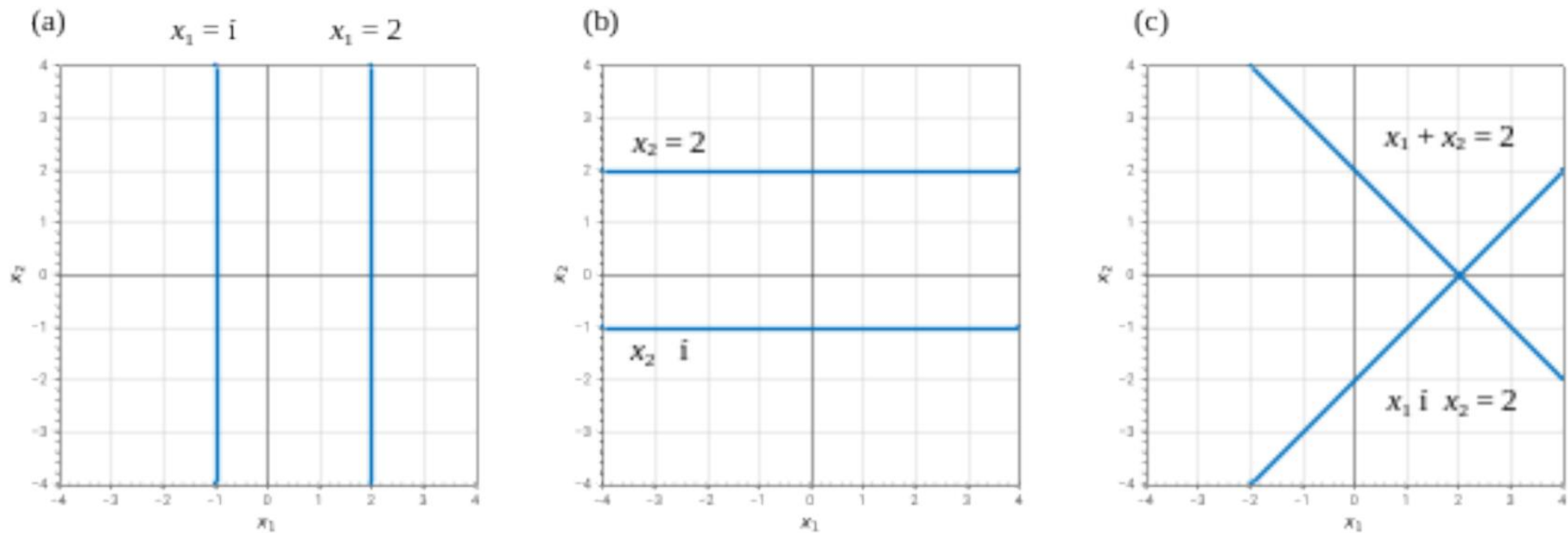


图 1. 几何视角看二元一次方程

Figure 2 · The Intersection of Two Lines Is the Solution

- The red line $x_1 + x_2 = 35$ and the blue line $2x_1 + 4x_2 = 94$ meet at $(23, 12)$ — the solution to the chicken-rabbit puzzle

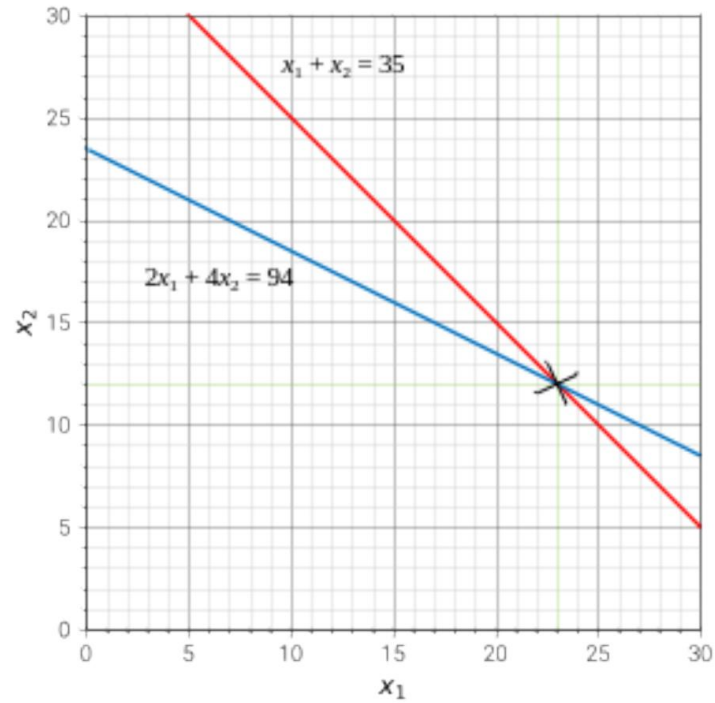


图 2. 图解鸡兔同笼问题

Figure 3 · Rewriting the System as $Ax=b$

- Orange is coefficients, pink is unknowns, blue is constants — stacking the two equations gives (c), the matrix form $Ax=b$

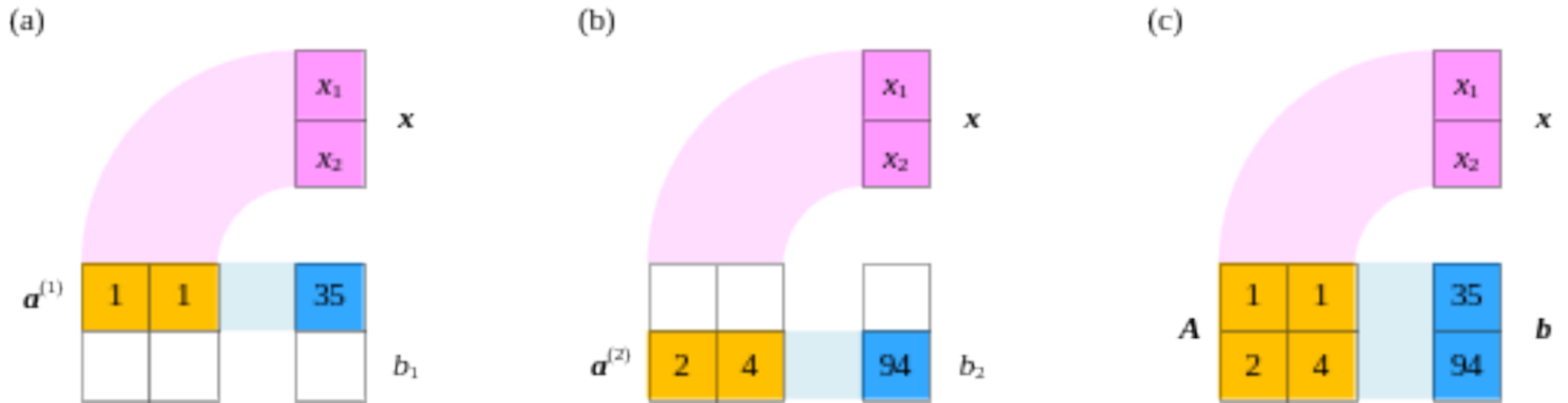


图 3. 把线性方程组写成矩阵乘法形式

Solving $Ax=b$: $x=A^{-1}b$

$$x = A^{-1}b$$

- A is called the coefficient matrix, b the vector of constant terms; when A is a square, invertible matrix, x follows directly from the formula above
- Here $\det(A)=2\neq 0$, so A is invertible; plugging in the numbers gives $x=[23, 12]^T$, matching the elimination result

Solving via the Inverse Matrix

Compute the inverse with `numpy.linalg.inv()`, then use `@` for matrix multiplication

- `A = np.array(...)`: the coefficient matrix — each column is an unknown, each row an equation
- `b = np.array(...)`: the constant vector — its two entries are the head count and foot count
- `A_inv = np.linalg.inv(A)`: invert **A** (check `det(A)` with `np.linalg.det()` first)
- `A_inv @ A` and `A @ A_inv`: both should equal the identity matrix
- `A_inv @ b`: gives the solution **x** — the number of chickens and rabbits

```
## 初始化
import numpy as np

## 鸡兔同笼系数矩阵
a A = np.array([[1, 1],
                [2, 4]])

## 常数列向量
b b = np.array([[35],
                [94]])

## 矩阵A的逆
c A_inv = np.linalg.inv(A)

# 验证
d A_inv @ A
  A @ A_inv

## 求解
e A_inv @ b
```

Why Express Linear Systems as Matrix Multiplication?

More compact: avoids repeatedly writing out every variable, especially with many unknowns

Richer information: the rank of $\mathbf{A}$ quickly reveals unique, no, or infinitely many solutions

Generalizes further: overdetermined systems, eigendecomposition, and linear transforms all build on matrix notation

More efficient computation: matrix operations run far faster and more precisely than solving equations one by one

Figure 4 · Unique, None, or Infinitely Many Solutions

- (a) the lines coincide $\rightarrow$ infinitely many solutions; (b) the lines are parallel $\rightarrow$ no solution; (c) the lines cross $\rightarrow$ a unique solution

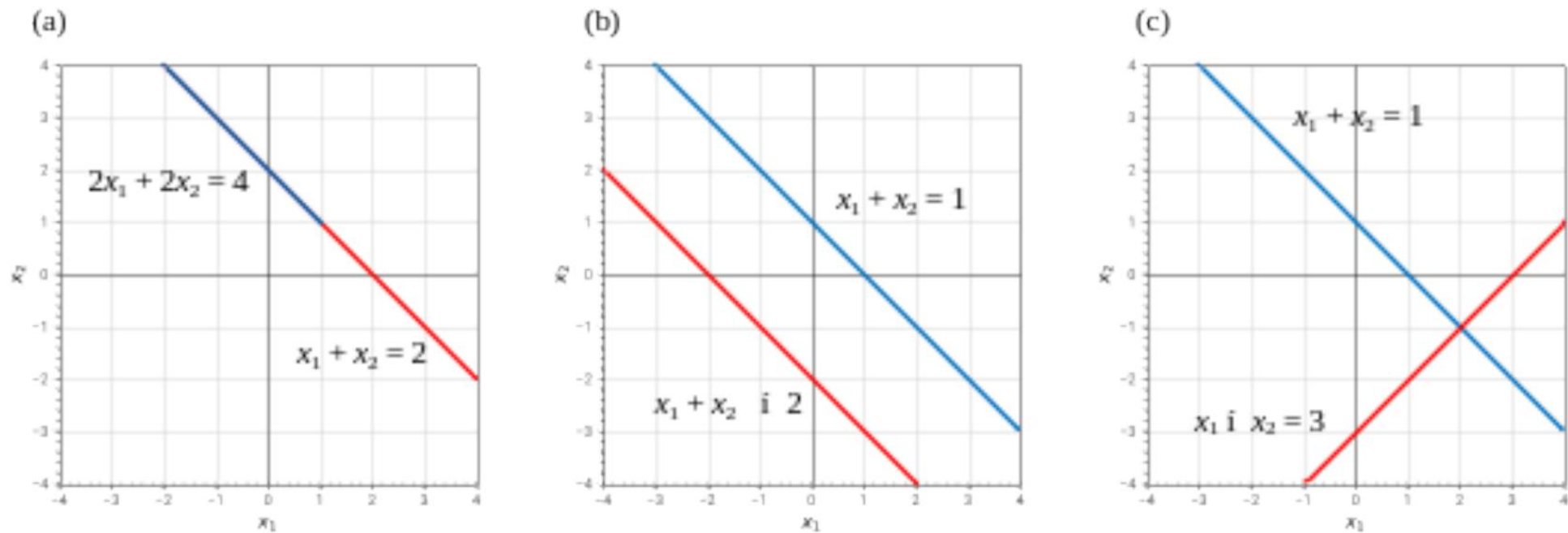


图 4. 不同解的数量

$\det(\mathbf{A})$ and the Type of Solution

$\det(\mathbf{A})=0 \Rightarrow$ no solution or infinitely many; $\det(\mathbf{A})\neq 0 \Rightarrow$ a unique solution

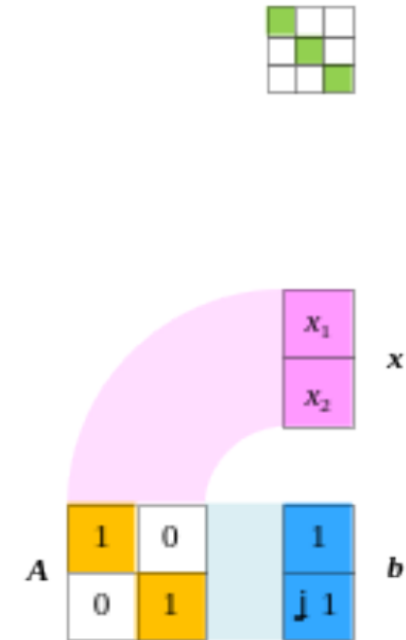
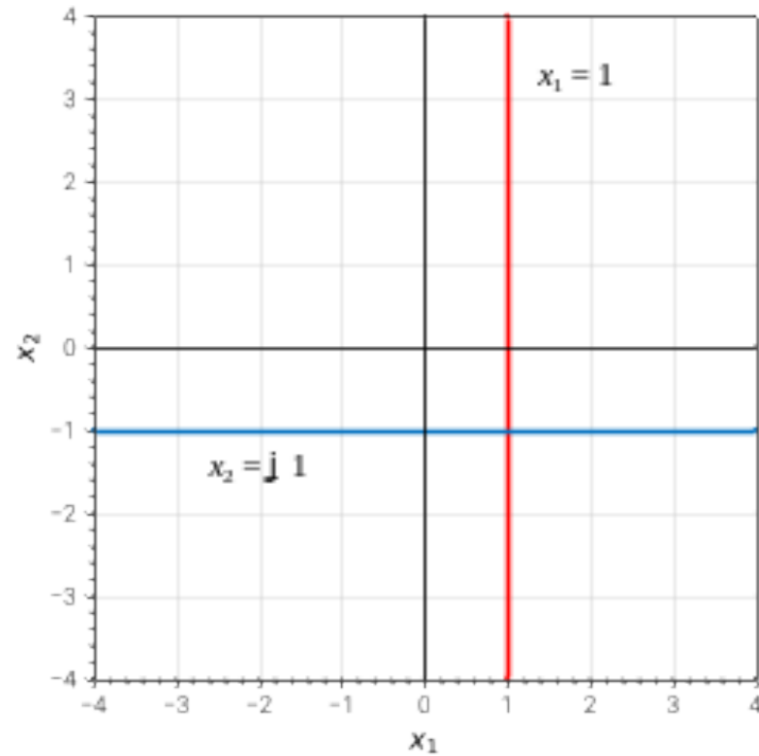
- Figure 4(a): the two equations are equivalent, $\det(\mathbf{A})=0$, infinitely many solutions; Figure 4(b): parallel lines never meet, $\det(\mathbf{A})=0$, no solution
- Figure 4(c): the lines cross, $\det(\mathbf{A})=-2\neq 0$, $\mathbf{A}$ is invertible, so the system has a unique solution

Three Elementary Row Operations

- ① Row swapping: exchange two rows of a matrix
- ② Row scaling: multiply an entire row by a nonzero scalar
- ③ Row sum: add a nonzero multiple of one row to another

Figure 5 · The Identity Matrix Is "Perfect"

- $x_1=1$ and $x_2=-1$ are already the solution — no extra computation needed at all



When $A=I$, $x=b$

$$A = I \Rightarrow x = A^{-1}b = b$$

- The whole goal of solving $Ax=b$ is to reduce the square matrix A to the identity matrix I

Figure 6 · Swap Rows, A Becomes a Permutation Matrix

- $A = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$, $\det(A) = -1$; left-multiplying by A has exactly the effect of "swapping rows"

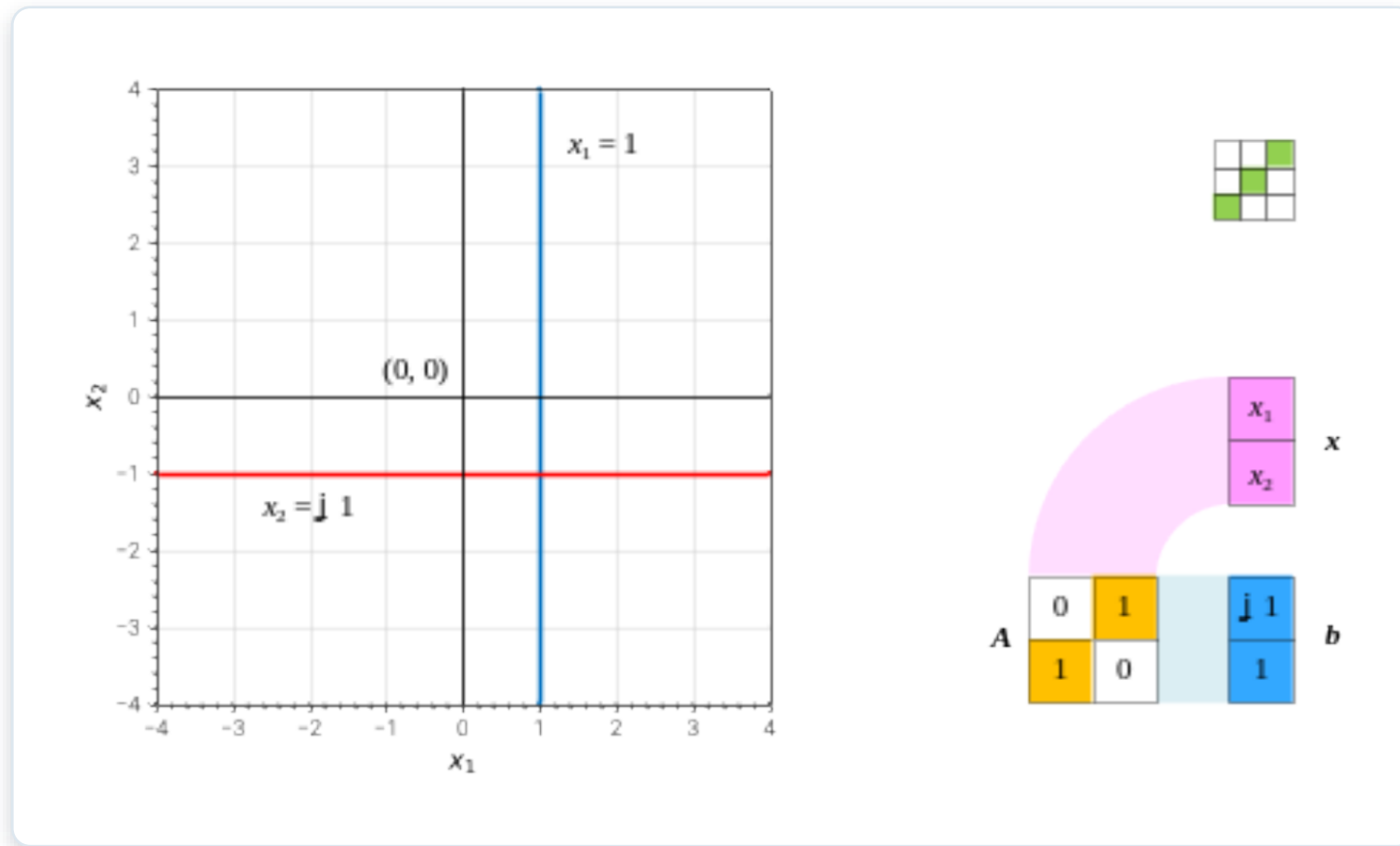


Figure 7 · Row Swapping Only Reorders the Lines

- Written $R_1 \leftrightarrow R_2$; row swapping doesn't affect the solution — the two lines themselves never change

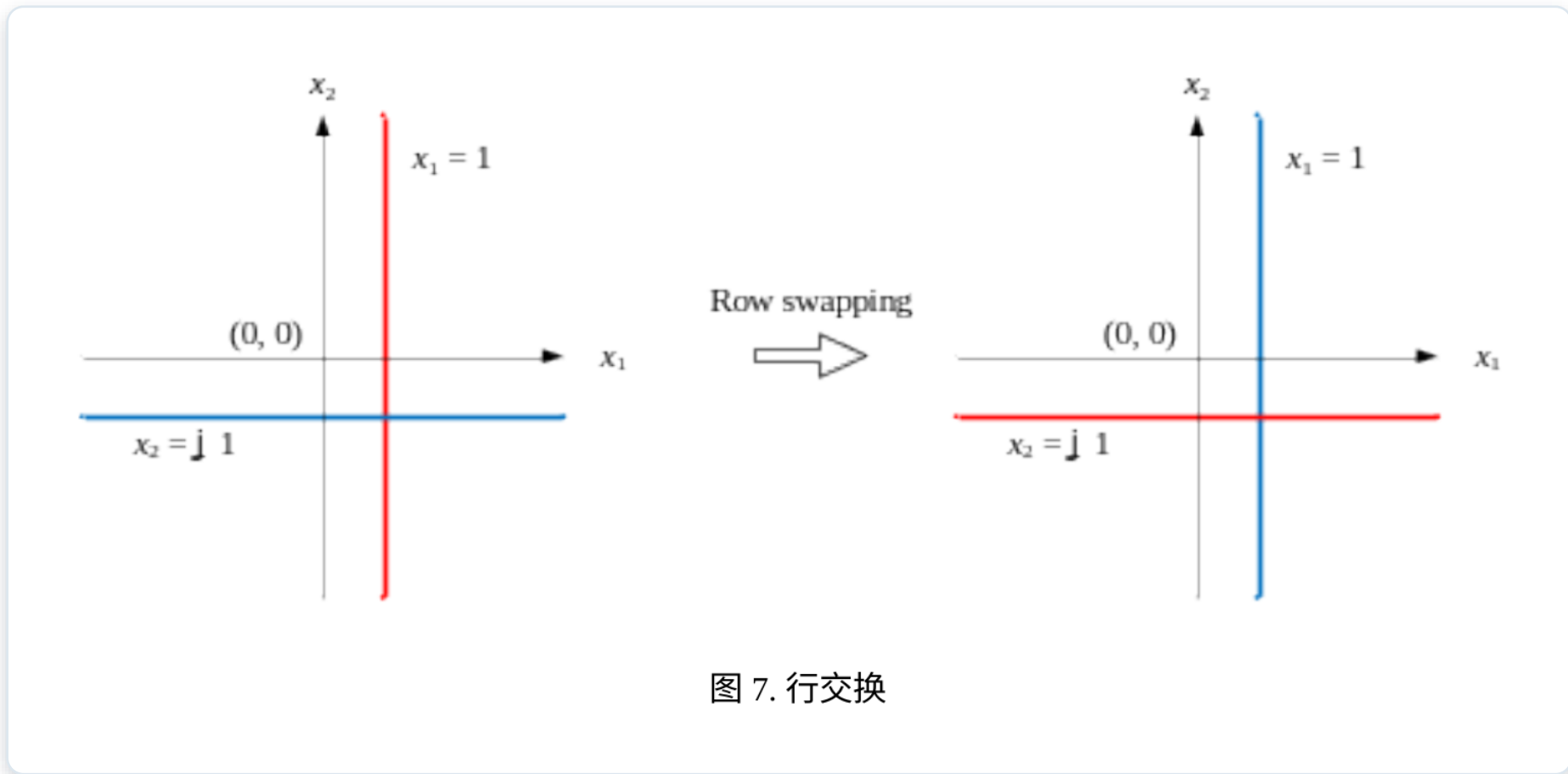


图 7. 行交换

Figure 8 · A Diagonal Matrix Is Already Close

- $A = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$, $\det(A) = 6$, invertible; $2x_1 = 2$ already looks close to the solution $x_1 = 1$

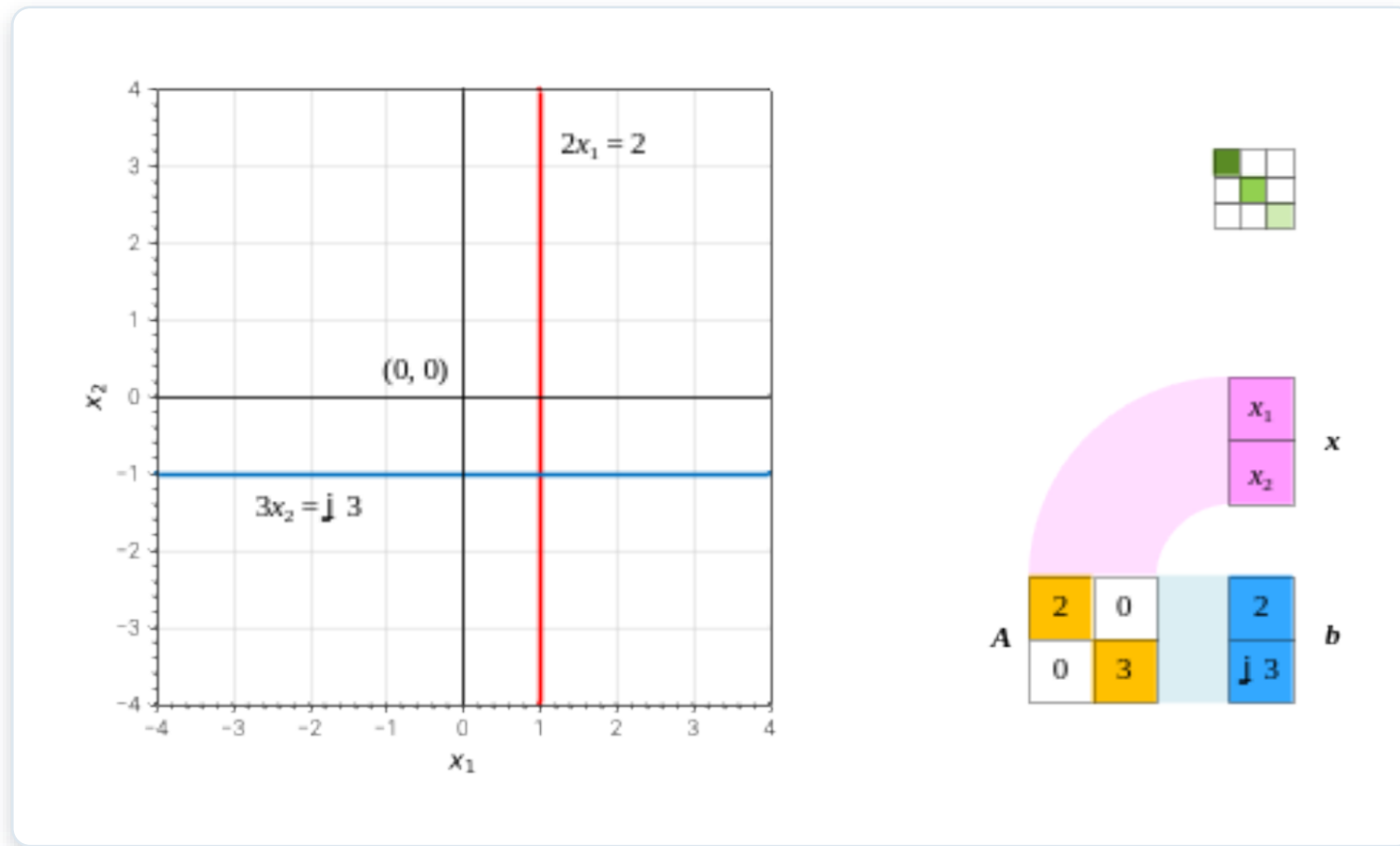


Figure 9 · Row Scaling Doesn't Change a Line's Position or Shape

- Written $R_1/2 \rightarrow R_1$, $R_2/3 \rightarrow R_2$; these two steps are exactly the $1/2$ and $1/3$ hiding inside $\mathbf{A}$'s inverse $\mathbf{A}^{-1}$

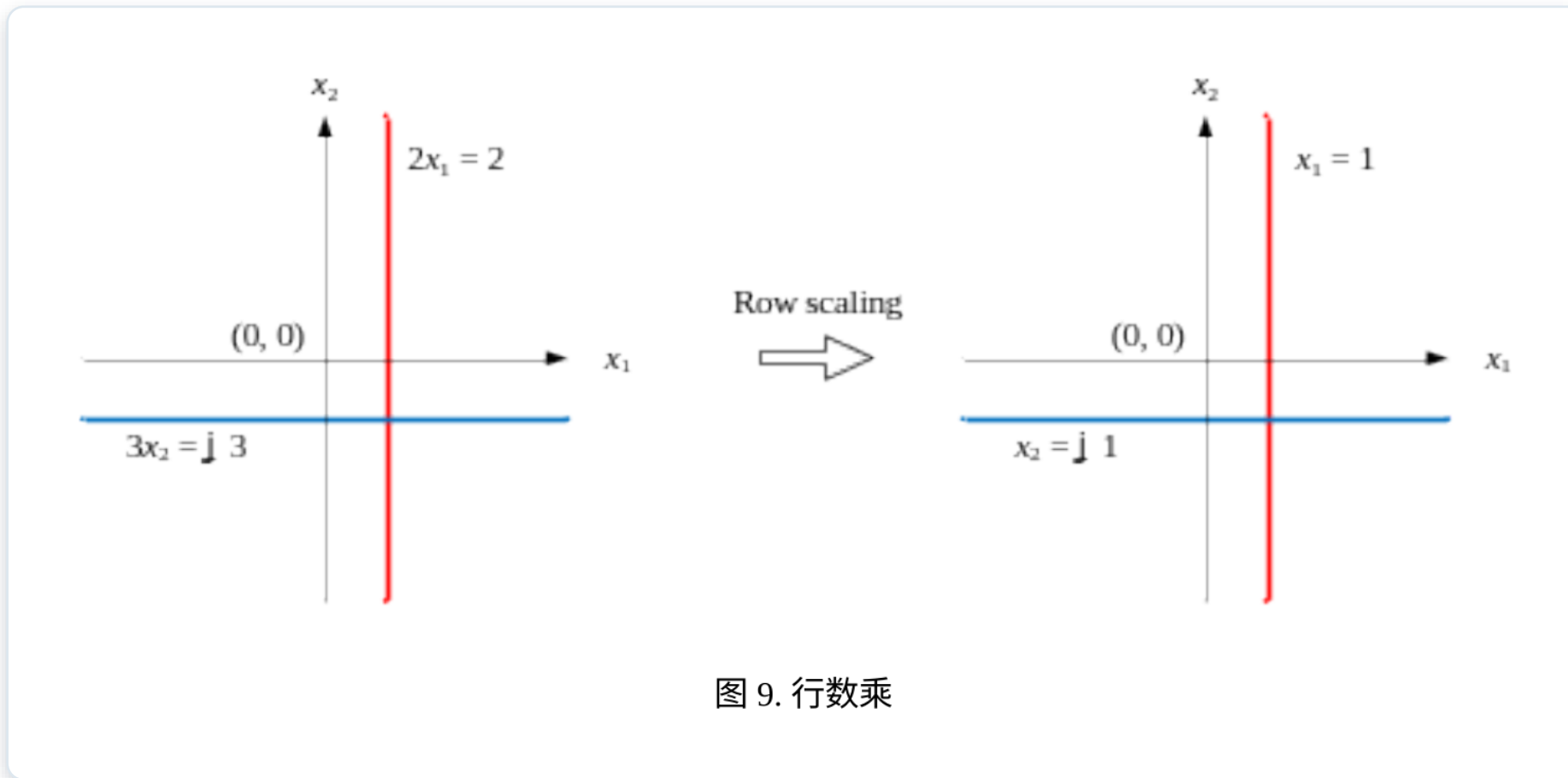


Figure 10 · An Upper-Triangular Matrix, Step by Step

- $A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$, $\det(A) = 1$, invertible; the second row R_2 already gives the solution for x_2 directly

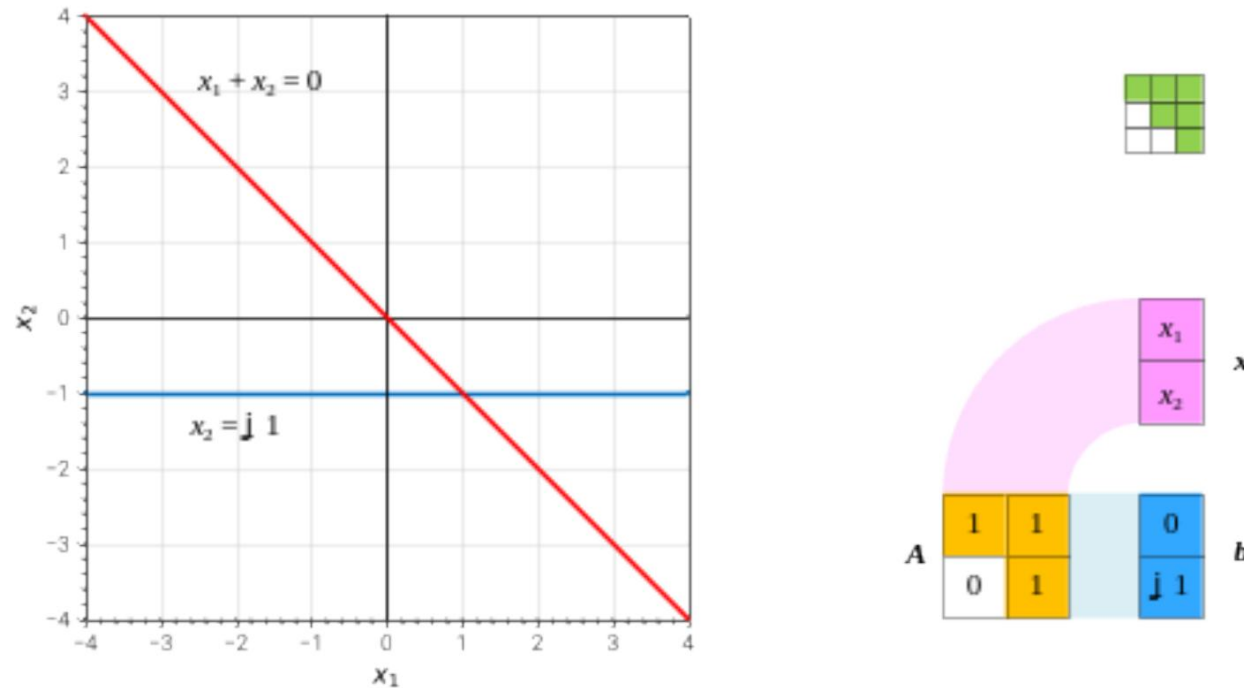


图 10. 方阵 A 是上三角矩阵

Figure 11 · Row Sum Changes a Line's Shape and Position

- Written $R_1 - R_2 \rightarrow R_1$; subtracting the second row from the first, and replacing the first, solves for x_1 — and reduces $\mathbf{A}$ to the identity $\mathbf{I}$

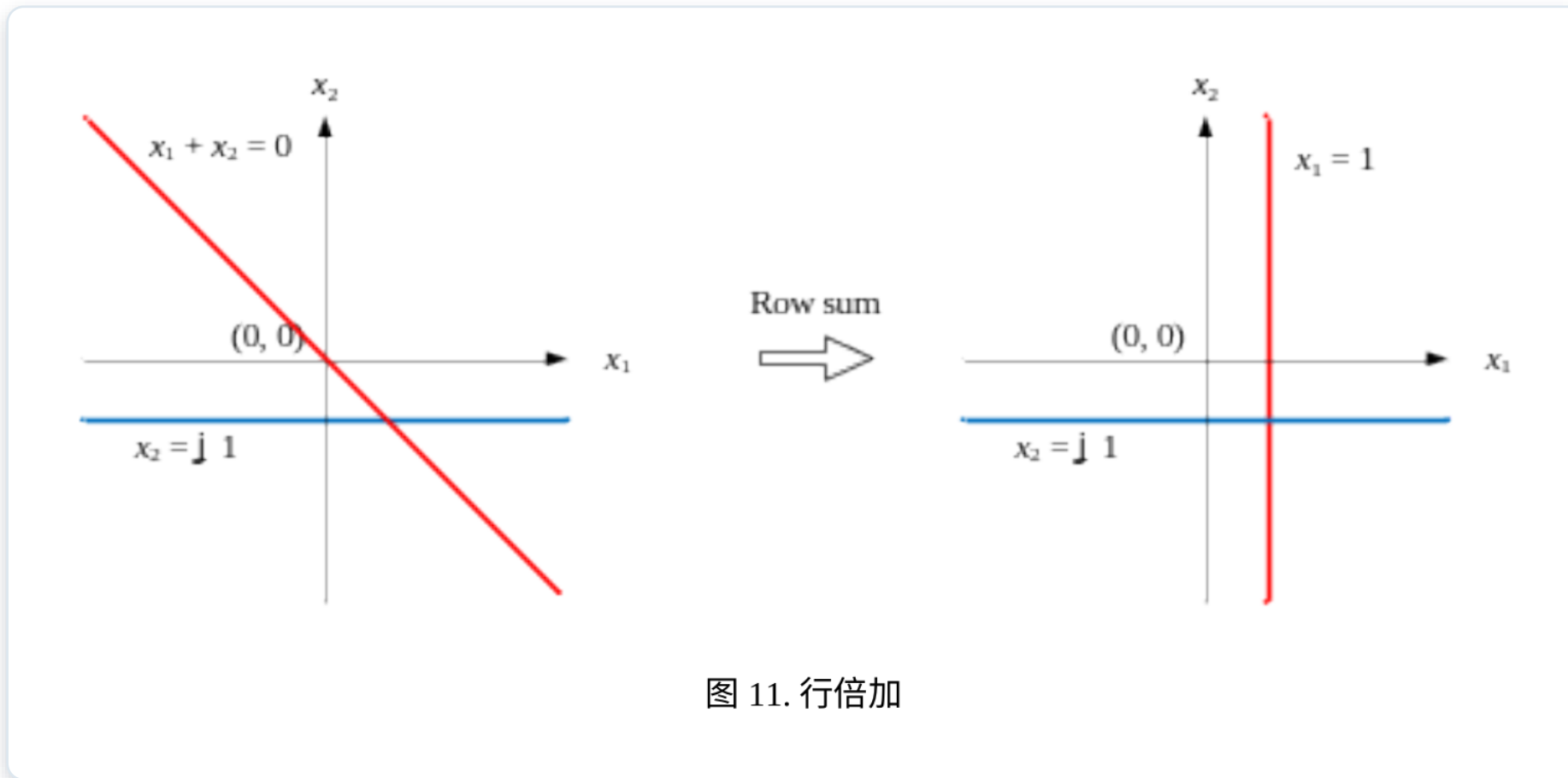


Figure 12 · A Lower-Triangular Matrix, the Same Idea

- $A = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$, $\det(A) = 1$, invertible; written $R_2 - R_1 \rightarrow R_2$, this reduces A to the identity matrix I

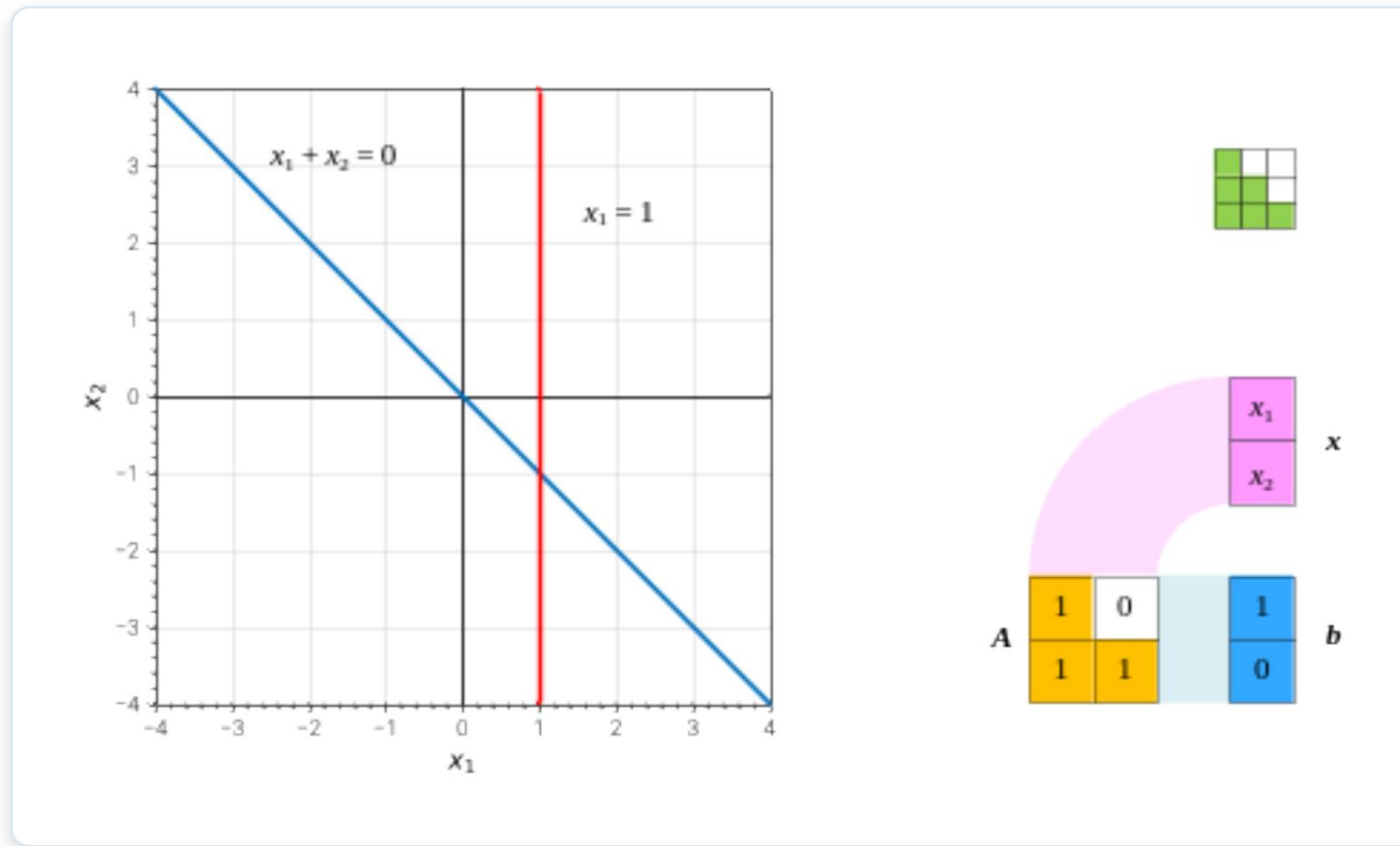


Figure 13 · $Ax=b$ Has a Unique Solution

- $A=[[1,-1],[2,2]]$, $\det(A)=8$; reduce to upper (or lower) triangular, then diagonal, then finally the identity I

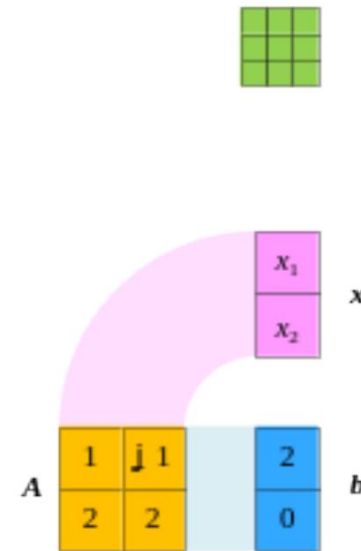
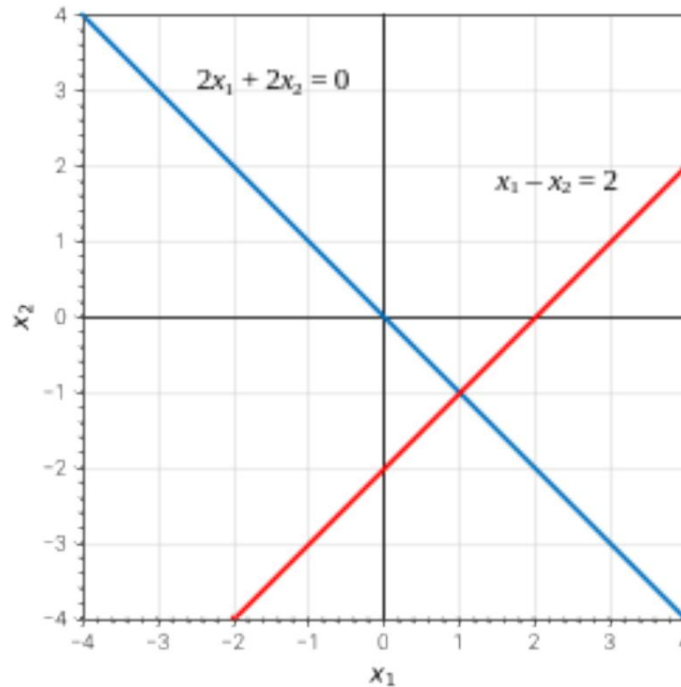


图 13. 方阵 A 行列式不为 0, $Ax = b$

Homogeneous Linear System

$$Ax = \mathbf{0}$$

- A linear system whose constant terms are all zero ($\mathbf{b}$ is the zero vector $\mathbf{0}$) is called a homogeneous linear system
- Both lines pass through the origin; when $\det(\mathbf{A}) \neq 0$, the origin $(0, 0)$ is the unique solution

Figure 14 · Both Lines Pass Through the Origin

- The intersection of the two lines is the solution to the homogeneous system — here $\det(\mathbf{A}) \neq 0$, so the unique solution is the origin $(0, 0)$

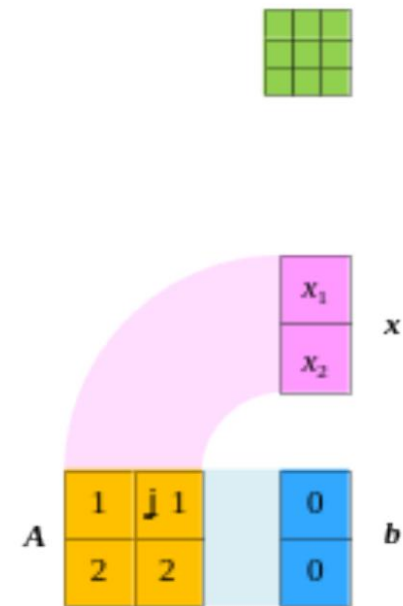
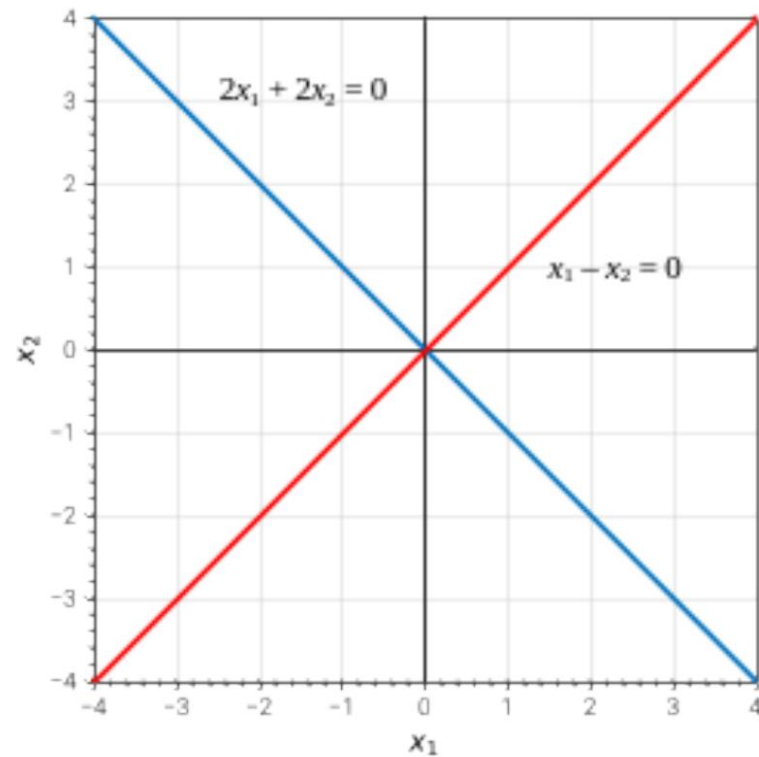


Figure 15 · The Lines Coincide, Infinitely Many Solutions

- A 's two lines completely coincide; $\det(A)=0$, A has no inverse, and $Ax=b$ has infinitely many solutions

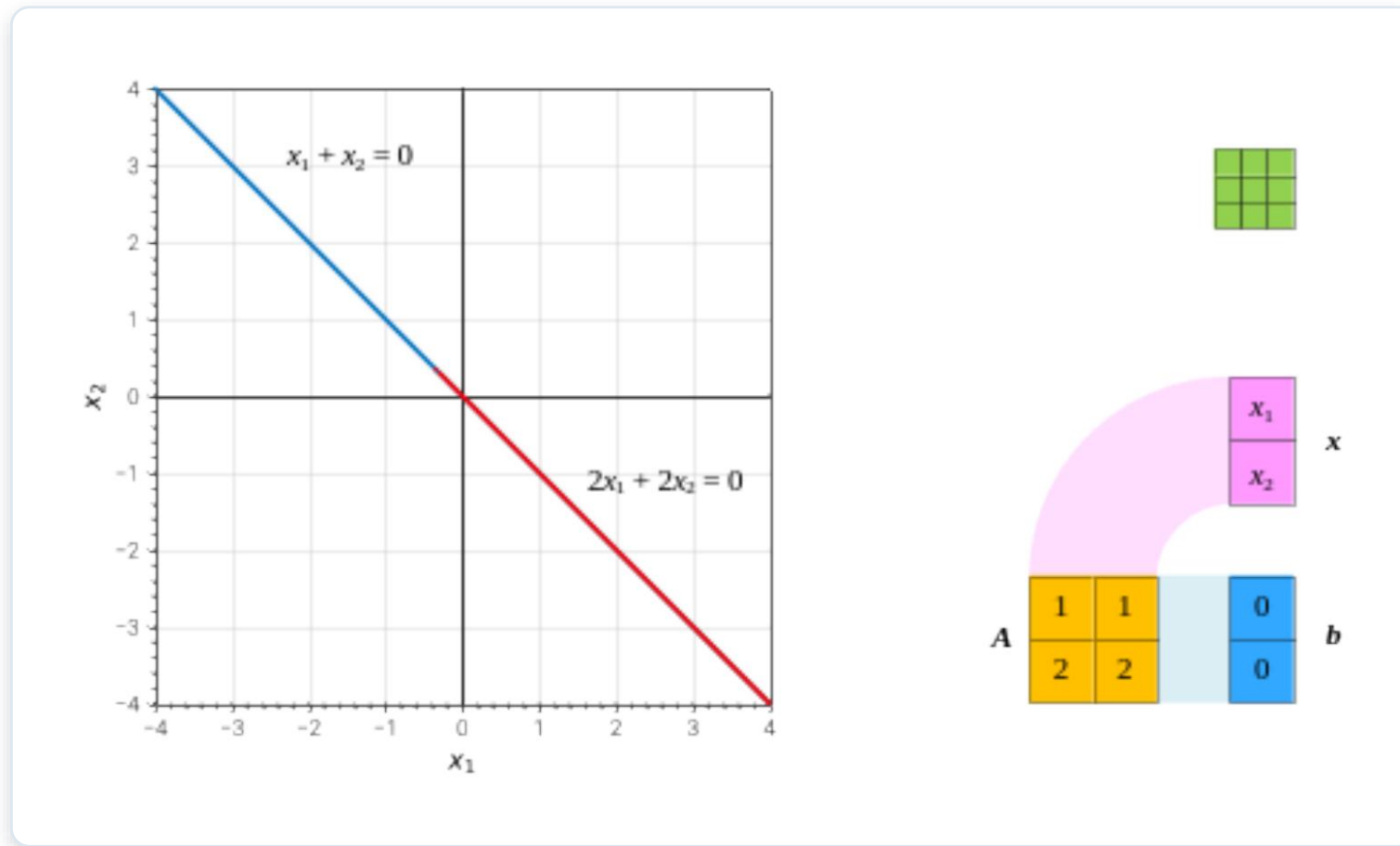
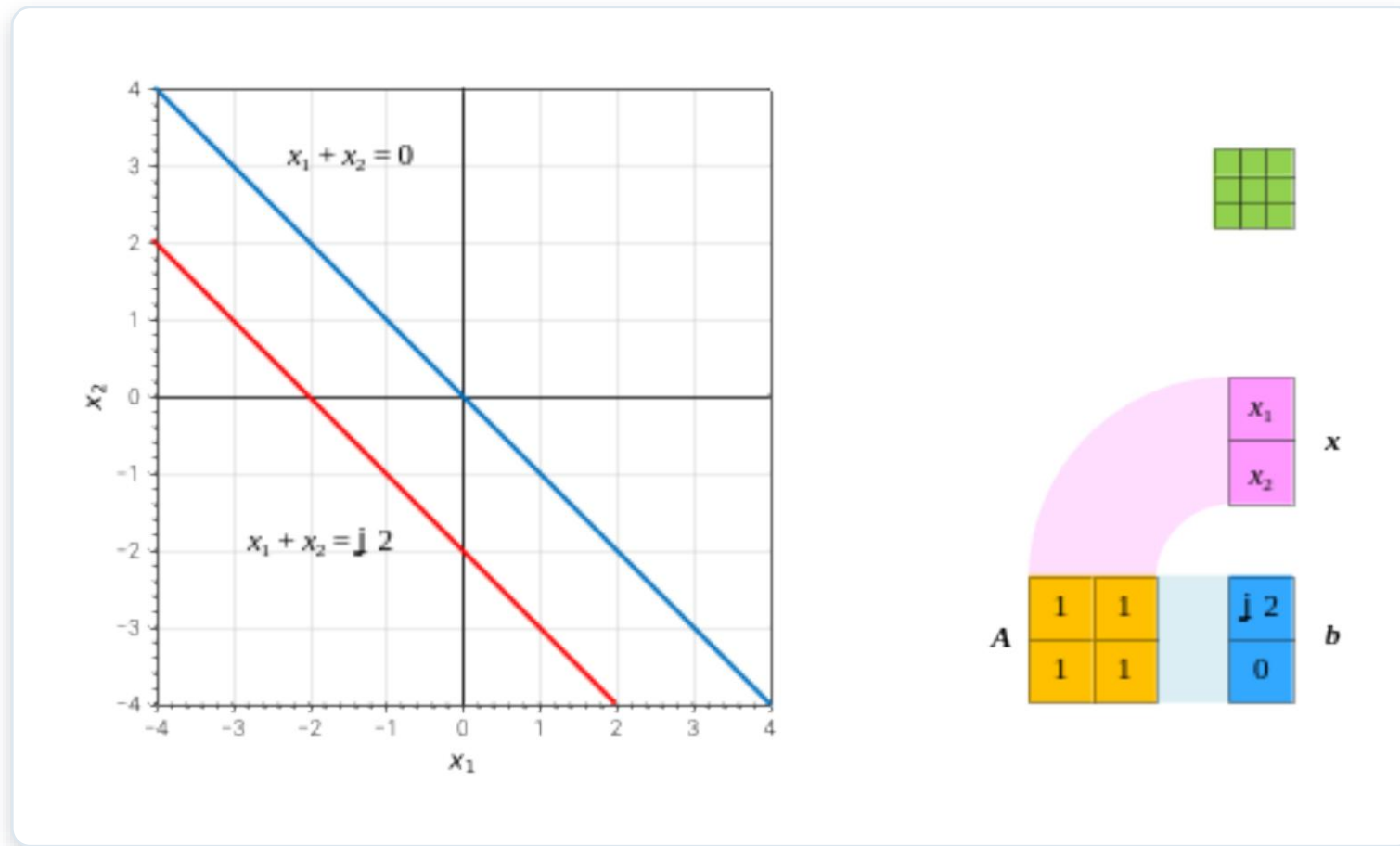


Figure 16 · The Lines Are Parallel, No Solution

- A 's two lines are parallel; $\det(A)=0$, A has no inverse, and $Ax=b$ has no solution



From Intersecting Lines to Matrices and Row Operations

$\det(\mathbf{A}) \neq 0 \Leftrightarrow \text{unique solution}; \det(\mathbf{A}) = 0 \Leftrightarrow \text{no solution or infinitely many}$

- The core idea for solving $\mathbf{Ax}=\mathbf{b}$: use elementary row operations (row swap, row scaling, row sum) to reduce $\mathbf{A}$ to the identity matrix $\mathbf{I}$ step by step
- The determinant $\det(\mathbf{A})$ decides the type of solution: nonzero means unique, zero means no solution or infinitely many — which one depends further on $\mathbf{b}$

Key Terms in This Section

Linear Equation in Two Variables $a_1x_1 + a_2x_2 = b$

Coefficient Matrix A — built from the coefficients of each equation

Determinant $\det(A)$ — tests whether square matrix A is invertible

Permutation Matrix A special square matrix that performs "row swapping"

Homogeneous Linear System A system whose constant vector b is entirely zero

System of Linear Equations A set of linear equations solved together

Vector of Constant Terms b — the constants on the right-hand side

Elementary Row Operations Row swapping, row scaling, row sum

Diagonal Matrix A square matrix with nonzero entries only on the diagonal

Inverse Matrix A^{-1} — the key tool for solving $x = A^{-1}b$

Further Reading

Further Reading · `numpy.linalg.solve()`

LA_07_01_02.ipynb solves the chicken-rabbit puzzle directly with `solve()`, no manual inverse needed

Further Reading · Symbolic Solving with SymPy

LA_07_01_03.ipynb solves the linear system exactly, using symbolic computation

Further Reading · Visualizing Solutions with Contours

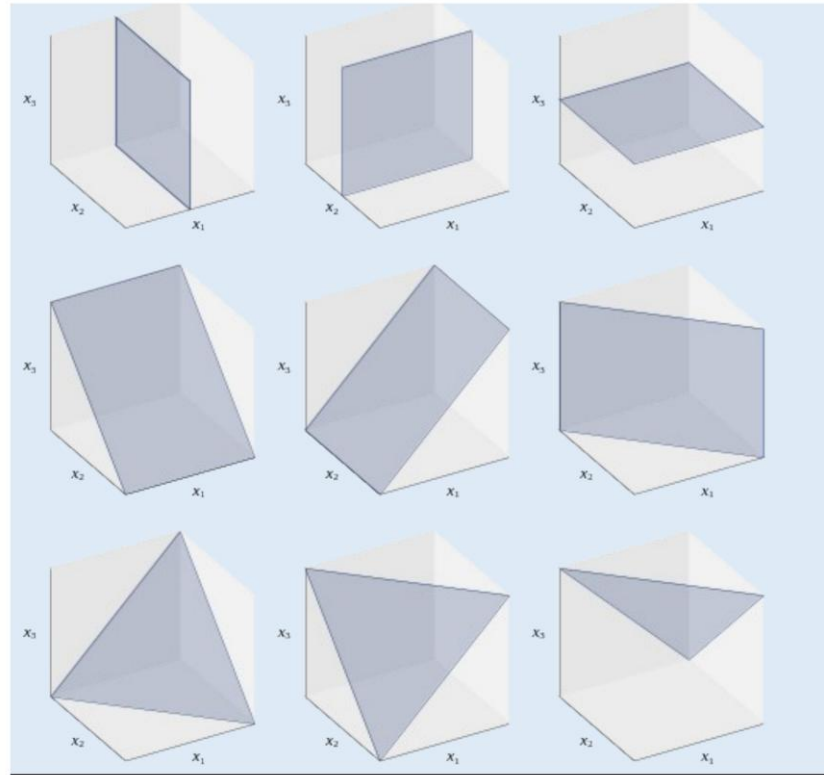
LA_07_01_04.ipynb visualizes a linear system's solution via contour plots

Open-Source Repository

Code and figures for the full book: <https://github.com/Visualize-ML>

Figure 17 · Nine Planes, Paired with Exercise Q3

- A three-variable linear equation $a_1x_1+a_2x_2+a_3x_3=b$ traces a plane in 3D space; the signs and zeros of its coefficients set the plane's orientation



Exercises Q1 – Q8

Q1 Write each of the following two linear systems in matrix multiplication form: $x_1+x_2=4$, $2x_1+4x_2=12$; $x_1+x_2=2$, $x_1-x_3=1$, $x_2-x_3=0$.

Q3 A three-variable linear equation traces a plane in 3D space; for each of the 9 plane images, determine the sign or zero-ness of coefficients a_1 , a_2 , a_3 , and the value of b .

Q5 Use `numpy.linalg.inv()` to invert the invertible matrices from Q4, and verify with `@` that `A_inv@A` and `A@A_inv` both equal the identity matrix.

Q7 Use `matplotlib` to plot the two lines $x_1+x_2=35$ and $2x_1+4x_2=94$, marking their intersection.

Q2 Write Python code to solve the two systems in Q1, using `numpy.linalg.solve()` or the inverse-matrix method.

Q4 Use `numpy.linalg.det()` to test whether these matrices are invertible: `[[1,1],[2,2]]`, `[[1,1],[1,-1]]`, `[[2,0],[0,3]]`, `[[0,1],[1,0]]`.

Q6 Write a function `solve_2x2(A, b)` that solves a 2×2 linear system via the inverse-matrix method, printing a warning when `det(A)=0`.

Q8 Use `numpy` to verify that left-multiplying any 2D vector by `[[0,1],[1,0]]` swaps its two components.

Exercises Q9 – Q16

Q9 Write a function `row_swap(A, i, j)` that swaps rows i and j of matrix A .

Q11 Write a function `row_add(A, i, j, k)` that adds k times row j to row i (row sum).

Q13 Use numpy to verify that the determinant of an upper- or lower-triangular matrix equals the product of its diagonal entries.

Q15 Randomly generate 100 2×2 matrices with numpy, and count what fraction have determinant 0 (non-invertible).

Q10 Write a function `row_scale(A, i, k)` that multiplies row i of matrix A by a nonzero scalar k (row scaling).

Q12 Combine `row_swap`, `row_scale`, and `row_add` to hand-write Gaussian elimination, reducing a 2×2 invertible matrix to the identity.

Q14 Construct a 3×3 upper-triangular matrix and a 3×3 lower-triangular matrix, and confirm Q13's result with `numpy.linalg.det()`.

Q16 Use numpy to verify that the homogeneous system $\mathbf{Ax}=\mathbf{0}$ has only the zero solution $\mathbf{x}=[0,0]^T$ when $\det(A) \neq 0$.

Exercises Q17 – Q25

Q17 Construct a 2×2 homogeneous system with $\det(A)=0$, and use `numpy.linalg.matrix_rank()` to check whether it has infinitely many solutions.

Q19 Write a function `classify_2x2_system(A, b)` that uses $\det(A)$ and the ranks of A and $[A|b]$ to automatically classify a system as unique, no, or infinitely many solutions.

Q21 Use `sympy.linsolve()` to solve the chicken-rabbit puzzle symbolically, and compare with the numpy numerical solution.

Q23 Use `numpy.linalg.cond()` to compute the condition number of several coefficient matrices A from this section, and consider how condition number relates to numerical stability.

Q25 Comprehensive exercise: pick any three-variable linear system, write it in matrix form $\mathbf{Ax}=\mathbf{b}$, and use Python to determine its type of solution (unique / infinite / none).

Q18 Use matplotlib to plot the "infinitely many solutions" and "no solution" cases where $\det(A)=0$ (compare Figures 15 and 16).

Q20 Test the function from Q19 on the four systems behind Figures 4, 13, 15, and 16, and confirm the classifications are correct.

Q22 Adapt the chicken-rabbit puzzle: a cage holds turtles and cranes, 80 heads and 236 legs in total — how many of each? Solve it in Python.

Q24 Write a function `plot_2x2_system(A, b)` that automatically plots the two lines of any 2×2 linear system and their intersection (if one exists).

10 Prompts to Explore with AI

- 1 Explain why a linear equation in two variables graphs as a line, while a linear equation in three variables graphs as a plane.
- 2 Prove that when the determinant $\det(\mathbf{A})$ of a square matrix $\mathbf{A}$ is zero, the linear system $\mathbf{Ax}=\mathbf{b}$ has either no solution or infinitely many.
- 3 Explain which special matrices correspond to "row swapping," "row scaling," and "row sum" among the elementary row operations.
- 4 Use Python to demonstrate reducing a general 2×2 invertible matrix to the identity matrix, step by step, via elementary row operations.
- 5 Explain why the identity matrix is the "ultimate goal" when solving $\mathbf{Ax}=\mathbf{b}$.

10 Prompts to Explore with AI

6 Explain the relationship between the solutions of a homogeneous system $\mathbf{Ax}=\mathbf{0}$ and a non-homogeneous system $\mathbf{Ax}=\mathbf{b}$.

7 Give a real-world example that can be modeled as a system of linear equations in two variables, and express it in matrix form.

8 Explain why engineering and data science prefer the matrix form $\mathbf{Ax}=\mathbf{b}$ over solving equations one at a time.

9 Use NumPy to compare `numpy.linalg.inv()` and `numpy.linalg.solve()` for solving the same linear system, and compare their computational efficiency.

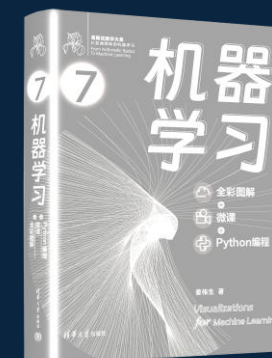
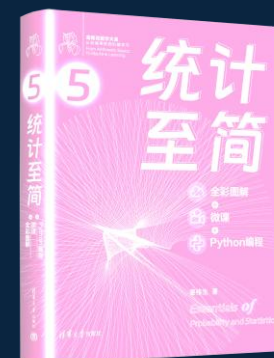
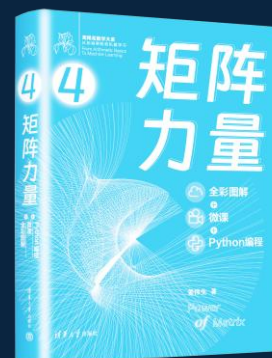
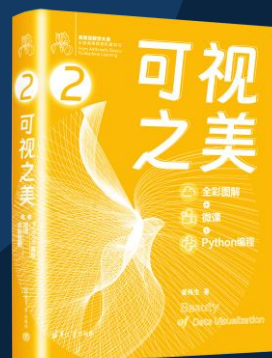
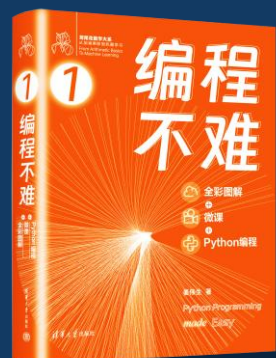
10 Design a small Python project: given the coefficients of any 2×2 linear system, automatically classify its solution type and visualize the two lines.

数学

数学基础

线性代数

概率统计



Python编程

数据可视化

工具

数据分析
图和网络

回归、降维
分类、聚类

实践

